

Hikru Insight (Proyecto de la Liga Hikru)

Documento de Diseño (v1.0)

Autores: Marco Angulo, Mauro Carlucci

Fecha: 18/12/2025

Repositorio: <https://github.com/maurocarlucci/LigaHikru>

1. Resumen ejecutivo

Hikru Insight es un asistente inteligente para consulta de documentos internos que reduce el tiempo que los equipos invierten en buscar políticas, procedimientos y respuestas operativas dentro de archivos dispersos (PDF, TXT, MD). El sistema permite tres capacidades principales: **preguntar en lenguaje natural** y recibir una respuesta **generada por IA con citas** a los documentos fuente, **buscar fragmentos relevantes** mediante búsqueda semántica e híbrida (texto + vectorial), retornando resultados con enlaces al origen y un manejo **interno de documentos**.

La solución se implementa como un MVP (Producto Mínimo Viable) orientado a una demostración clara y reproducible. La arquitectura utiliza **Azure OpenAI** para embeddings y generación de respuestas, **Azure AI Search** para indexación y recuperación semántica (vectorial) combinada con texto, y **Azure Blob Storage** como repositorio de documentos. A nivel de aplicación, el backend se desarrolla con **FastAPI (Python)** y el frontend con **React (Vite)** para una interfaz web profesional.

El beneficio central es operativo: en lugar de que una persona de soporte o RRHH tenga que abrir múltiples documentos, navegar, buscar dentro de los mismos y validar manualmente si la información encontrada está vigente, Hikru Insight ofrece una respuesta sintetizada **anclada** al contenido indexado y acompañada de **evidencias** (citas) que facilitan auditoría y confianza.

2. Problema, usuario y alcance

2.1 Problema a resolver

En equipos de soporte, recursos humanos y operaciones, es común que las respuestas a preguntas frecuentes (por ejemplo, “¿cuál es la política de vacaciones?”, “¿cómo se gestiona un reembolso?”, “¿qué pasos debo seguir para X proceso?”) existan ya en documentos internos. Sin embargo, en la práctica se presentan varios obstáculos:

1. **Fragmentación de la información:** las políticas y procedimientos se encuentran repartidos en distintos archivos, versiones y ubicaciones.

2. **Baja accesibilidad:** aunque la información esté disponible, localizarla requiere tiempo y depende de experiencia previa de dónde ubicarla.
3. **Costo de contexto:** una búsqueda manual obliga a leer secciones extensas, interpretar y resumir, incluso para preguntas simples.
4. **Riesgo de inconsistencia:** ante presión de tiempo, se responde con memoria o “conocimiento general”, sin certeza de que el documento vigente respalde la respuesta.
5. **Escalabilidad limitada:** cuando el volumen de documentos crece, el método manual deja de ser viable.

Este problema no es teórico: se refleja diariamente en tiempos de atención más altos, interrupciones constantes entre equipos, y pérdida de productividad por búsquedas repetitivas. En contextos organizacionales reales, la ausencia de una “fuente única consultable” produce fricción operativa y aumenta el riesgo de errores por desinformación.

2.2 Usuario final (personas y necesidades)

Usuarios principales

- **Soporte interno / Service Desk:** requiere respuestas rápidas, con respaldo, para atender solicitudes recurrentes.
- **RRHH:** necesita citar políticas de manera precisa y consistente (vacaciones, permisos, beneficios, lineamientos).
- **Operaciones:** consulta procedimientos, checklists, flujos operativos y reglas internas.

Necesidades clave

- Encontrar información **confiable y rastreable**.
- Disminuir el tiempo de búsqueda.
- Obtener un resumen **claro** sin perder referencia al origen.
- Poder validar la respuesta consultando la cita/documento original.

2.3 Alcance del MVP

El MVP se diseña para entregar valor con flujos simples, medibles y demostrables:

Flujo 1: Preguntar (Q&A con citas)

- El usuario escribe una pregunta.
- El sistema recupera fragmentos relevantes y genera una respuesta.
- La respuesta incluye **citas** a los documentos usados (nombre del documento y página si aplica).

Flujo 2: Buscar (búsqueda de fragmentos + enlace al origen)

- El usuario escribe un término o frase.
- El sistema retorna resultados relevantes con fragmentos textuales y enlace al documento (URL de Blob).

Flujo 3: Gestión de documentos (mínimo necesario para alimentar el sistema)

- Subir documentos (PDF/TXT/MD).
- Listar documentos cargados.
- Eliminar documentos.

Fuera de alcance (por decisión de MVP)

- OCR para PDFs escaneados (solo imagen).
- Integración con repositorios corporativos (SharePoint, Confluence, etc.).
- Analítica avanzada y monitoreo profundo.
- Control de acceso “completo” (roles avanzados, multi-tenant).
- Procesamiento asíncrono y colas para cargas masivas.

Nota: El “control de acceso básico” se implementará posteriormente. En este documento se reserva la sección correspondiente, pero se deja sin contenido hasta finalizar la implementación.

3. Arquitectura en Azure

3.1 Vista general

La solución se compone de tres capas principales:

- 1. Frontend Web (React + Vite)**
 - Interfaz para “Preguntar”, “Buscar” y “Documentos”.
 - Capa de presentación y experiencia de usuario.
- 2. Backend API (FastAPI + Python)**
 - Expone endpoints REST para chat, búsqueda e ingestión.
 - Orquesta el flujo RAG y los accesos a servicios Azure.
 - Valida entradas y gestiona respuestas estructuradas.
- 3. Servicios Azure**
 - **Azure OpenAI:** embeddings + chat completions.
 - **Azure AI Search:** indexación y recuperación híbrida (texto + vector).
 - **Azure Blob Storage:** almacenamiento de documentos y URLs fuente.
 - **Azure Storage SQL:** Manejo de usuarios de la aplicación para login y signup

3.2 Componentes y responsabilidades

Frontend

- **ChatView (Preguntar):** muestra historial de mensajes y citas agrupadas por documento.
- **SearchView (Buscar):** lista resultados con score, página y fragmento (manteniendo legibilidad).
- **UploadView (Documentos):** carga, lista y elimina documentos (con feedback y validación de tipos).
- **api.js:** capa de integración HTTP con el backend (base URL configurable por variable de entorno).

Backend

- **Routers**
 - `POST /api/chat/ask`: respuesta con citas.
 - `POST /api/chat/search`: búsqueda de fragmentos.
 - `POST /api/documents/upload`: carga e indexación.
 - `GET /api/documents/list`: listado en Blob.
 - `DELETE /api/documents/{filename}`: eliminación en Blob.
 - `POST /auth/chat/signup`: crear usuario
 - `POST /auth/chat/login`: ingresar con usuario existente
 - `POST /auth/chat/login-json`: token para mostrar la cuenta en el header
- **Servicios**
 - `AzureOpenAIService`: embeddings y chat.
 - `AzureSearchService`: búsqueda híbrida, carga de chunks al índice, transformación a resultados/citas.
 - `AzureBlobService`: upload, list, delete, generación de URL.
 - `DocumentProcessor`: extracción de texto, chunking y preparación para indexación.
 - `RAGService`: orquestación “preguntar” y “buscar”.
 - `AzureStorageSQL`: manejo de usuarios.

3.3 Estrategia RAG (Retrieval Augmented Generation)

El flujo de Preguntar utiliza un patrón RAG: primero recupera evidencia relevante del índice, luego genera una respuesta con ese contexto. Esto evita que el modelo responda con “conocimiento general” y obliga a basarse en fragmentos existentes.

Reglas principales del asistente (a nivel de prompt de sistema):

- Responder **solo** con base en el contenido de documentos recuperados. Saludos simples se responden de forma genérica, sin mencionar documentos.
- Preguntas fuera de alcance se rechazan educadamente.

- Las respuestas relacionadas deben incluir citas con formato consistente.

3.4 Índice en Azure AI Search

El índice vectorial/híbrido contiene documentos por “chunks” (fragmentos), con campos clave:

- Identificador único (`id`)
- Nombre de documento (`documentName`)
- Contenido del fragmento (`content`)
- Vector embedding (`contentVector`)
- Metadatos para trazabilidad (`chunkIndex`, `pageNumber`, `sourceUrl`)

La recuperación se realiza mediante:

- **Búsqueda textual** (relevancia por términos).
- **Búsqueda vectorial** (similitud semántica).
- Combinación/ranking para mejores resultados.

3.5 Consideraciones de despliegue (visión futura)

Para producción, una arquitectura típica incluiría:

- Hosting de frontend (Static Web Apps o similar).
- Hosting de backend (App Service / Container Apps).
- Gestión de secretos (Key Vault).
- Observabilidad (Application Insights).
- Restricciones de red (Private Endpoints, WAF/Front Door).

En el MVP, el despliegue se orienta a ejecución local y demostración funcional end-to-end.

4. Manejo de datos

4.1 Fuentes de datos y formatos soportados

El sistema opera sobre documentos que representan políticas y procedimientos internos. Para el MVP se soportan:

- **PDF** (texto extraíble)
- **TXT**
- **MD**

Los archivos residen en Blob Storage como fuente primaria. El índice de AI Search contiene fragmentos derivados (chunks) con embeddings.

4.2 Pipeline de ingestión (Upload → Index)

Paso 1: Carga del documento

- El usuario sube un archivo desde la UI.
- El backend valida extensión y procesa el contenido.

Paso 2: Almacenamiento

- Se sube el archivo a Blob Storage en un contenedor específico.
- Se genera una URL de referencia ([sourceUrl](#)) para trazabilidad y consulta.

Paso 3: Extracción de texto

- PDF: lectura por página (PyPDF2).
- TXT/MD: decodificación UTF-8 directa.

Paso 4: Chunking (fragmentación controlada)

Objetivo: crear fragmentos coherentes, suficientemente pequeños para buena recuperación, y con solapamiento para mantener contexto.

Características del enfoque actual:

- Tamaño objetivo de chunk alrededor de ~700 caracteres.
- Overlap para continuidad (~150 caracteres).
- Preservación de estructura (saltos de línea y listas) para no degradar el contenido.
- Separación por secciones cuando se detectan headers (numerados, mayúsculas, romanos).
- Split recursivo de secciones largas respetando párrafos, líneas y finales de frase.
- Detección de listas para evitar cortes a mitad de ítems.

Paso 5: Generación de embeddings

- Se generan embeddings por chunk.
- Se valida la dimensionalidad esperada (1536) antes de indexar.

Paso 6: Indexación

- Se indexa cada chunk como un documento en Azure AI Search con su vector y metadatos.
- El `id` se construye como `archivo_sanitizado + chunkIndex`.

4.3 Recuperación de datos para Preguntar y Buscar

Buscar

- El query se convierte en embedding.
- Se ejecuta búsqueda híbrida.
- Se retornan N resultados con fragmentos relevantes.

Preguntar

- La pregunta se convierte en embedding.
- Se buscan “N” fragmentos relevantes.
- Se construye un “contexto” con esos fragmentos.
- Se invoca el modelo de chat con instrucciones estrictas.
- Se retorna respuesta + citas.

4.4 Trazabilidad y citas

La confiabilidad del sistema depende de que el usuario pueda validar “de dónde salió” la respuesta. Por eso cada cita incluye:

- Nombre del documento
- Página (para PDFs cuando se dispone)
- URL de fuente (cuando corresponde)

Esto permite un flujo de trabajo realista: “respuesta rápida + evidencia” para auditoría interna o validación operativa.

5. Seguridad

5.1 Prácticas actuales (MVP)

- **Gestión de secretos:** credenciales en variables de entorno (.env) y no versionadas en Git.
- **Validación de entrada:** Pydantic valida estructuras de requests.
- **Validación de archivos:** solo extensiones permitidas (PDF/TXT/MD).
- **CORS:** configurado para permitir comunicación frontend-backend en entorno de demo (a endurecer para producción).

5.2 Control de Acceso Básico

Sistema de autenticación y autorización basado en JWT (JSON Web Tokens) con roles de usuario. Permite controlar el acceso a funcionalidades según el rol del usuario (administrador o usuario normal).

Componentes Principales

1. Autenticación (Autenticación)

- Registro de usuarios (`/api/auth/signup`)
- Login de usuarios (`/api/auth/login`, `/api/auth/login-json`)

- Generación de tokens JWT con información del usuario y rol
- Hash de contraseñas usando bcrypt (límite de 72 caracteres)

2. Autorización (Autorización)

- Sistema de roles: `admin` y `user`
- Dependencias FastAPI para proteger rutas:
 - `get\_current\_user`: Requiere autenticación válida
 - `require\_admin`: Requiere rol de administrador

3. Almacenamiento

- Base de datos: Azure SQL Database
- Tabla `users` con campos: `id`, `email`, `username`, `hashed\_password`, `role`, `created\_at`, `is\_active`
- Relaciones con `question\_history` y `search\_history` mediante `user\_id`

Tecnologías Utilizadas

- ****JWT****: Tokens firmados con HS256, expiración configurable (default: 24 horas)
- ****bcrypt****: Hash de contraseñas con salt automático
- ****FastAPI Dependencies****: Sistema de dependencias para protección de rutas
- ****OAuth2PasswordBearer****: Esquema OAuth2 para extraer tokens del header Authorization

Matriz de Permisos

Endpoint	Público	Usuario	Administrador
`POST /api/auth/signup`	✓	✓	✓
`POST /api/auth/login`	✓	✓	✓
`POST /api/chat/ask`	✓	✓	✓
`POST /api/chat/search`	✓	✓	✓
`GET /api/documents/list`	✗	✓	✓
`POST /api/documents/upload`	✗	✗	✓
`DELETE /api/documents/{filename}`	✗	✗	✓

Supuestos

1. Base de Datos Azure SQL

- La base de datos está configurada y accesible
- Las credenciales están en el archivo `.env`
- El firewall permite conexiones desde las IPs necesarias

2. Seguridad de Contraseñas

- Las contraseñas se hashean con bcrypt antes de almacenarse
- Las contraseñas nunca se transmiten en texto plano después del hash
- Límite de 72 caracteres debido a limitaciones de bcrypt

3. Tokens JWT

- Los tokens se almacenan en `localStorage` del navegador (frontend)
- Los tokens expiran después de 24 horas (configurable)
- El `JWT\_SECRET\_KEY` debe ser seguro y único en producción

4. Roles de Usuario

- Los usuarios nuevos siempre se crean con rol `user`
- Solo se pueden crear administradores usando el script `create\_admin.py`
- Los roles no se pueden cambiar desde la API (solo desde la BD o script)

5. Sesiones

- No hay refresh tokens (el usuario debe hacer login nuevamente al expirar)
- El token se valida en cada request protegida

5.3 Consideraciones de seguridad para producción a futuro (no implementadas)

- Restricción estricta de CORS por dominio.
- Autenticación (idealmente Azure AD / OAuth2).
- Autorización (roles: lector, editor, admin).
- Rate limiting y protección contra abuso.
- Auditoría (logs de acciones sobre documentos).
- Acceso a Blob mediante SAS tokens o contenedor privado.
- HTTPS obligatorio en despliegue.

6. Limitaciones y mejoras

6.1 Limitaciones actuales

1. **PDFs escaneados:** si el PDF es una imagen, no se extrae texto (no hay OCR).
2. **Sin filtros avanzados:** búsqueda sin facetas por fecha/tipo/categoría.
3. **Conversación limitada:** la Q&A no mantiene memoria conversacional robusta.
4. **Observabilidad mínima:** no hay métricas, trazas o alertas (MVP).

6.2 Mejoras propuestas (roadmap)

Corto plazo (quick wins)

- Endurecer CORS para permitir solo el frontend.
- Mensajes de error más accionables y consistentes.

- Mejor manejo de documentos grandes (feedback de progreso en UI).

Mediano plazo

- OCR con Azure Document Intelligence para PDFs escaneados.
- Soporte de formatos adicionales (DOCX, PPTX).
- Filtros de búsqueda (documento, rango de fechas, página).
- Vista previa de documentos dentro del frontend.

Largo plazo (producción)

- Azure AD + RBAC.
- Key Vault para secretos.
- Application Insights (trazas, métricas, logs).
- Procesamiento asíncrono (colas) para cargas masivas.
- Optimización de costos: batching, caching, y estrategias de reducción de tokens.

7. Apéndice: API (referencia corta)

Chat

- `POST /api/chat/ask`
- `POST /api/chat/search`

Documentos

- `POST /api/documents/upload`
- `GET /api/documents/list`
- `DELETE /api/documents/{filename}`

Autorización

- `POST /auth/chat/signup`
- `POST /auth/chat/login`
- `POST /auth/chat/login-json`

(La especificación completa está disponible en Swagger [/docs.](#))